Rapport Technique

Assistant intelligent de recommandation d'événements culturels

Projet Puls-Events

Système RAG (Retrieval-Augmented Generation)

Jonathan FERNANDEZ

Projet 7 — Formation Ingénieur IA — OpenClassrooms

Mars 2026

Table des matières

1. Objectifs du projet 4
1.1. Contexte 4
1.2. Problématique 4
1.3. Objectif du POC 4
1.4. Périmètre 4
2. Architecture du système 4
2.1. Schéma global (UML) 4
2.2. Technologies utilisées 5
3. Préparation et vectorisation des données 5
3.1. Source de données 5
3.2. Nettoyage 5
3.3. Chunking 6
3.4. Embedding 6
4. Choix du modèle NLP 6
4.1. Modèle cible : Mistral AI 6
4.2. Fallback : Google Gemini 6
4.3. Critères de sélection 6
4.4. Prompting 7
4.5. Limites du modèle 7
5. Construction de la base vectorielle 7
5.1. FAISS 7
5.2. Stratégie de persistance 7
5.3. Métadonnées associées 8
6. API et endpoints exposés 8
6.1. Framework 8
6.2. Endpoints 8
6.3. Exemple d'appel et de réponse 8
6.4. Gestion des erreurs 9
6.5. Tests 9
7. Évaluation du système 9
7.1. Jeu de test annoté 9
7.2. Évaluation manuelle 10
7.3. Évaluation automatisée (Ragas) 10
7.4. Analyse qualitative 10
8. Recommandations et perspectives 11
8.1. Ce qui fonctionne bien 11
8.2. Limites du POC 11
8.3. Améliorations possibles 11
9. Organisation du dépôt GitHub 12
10. Annexes 13

10.1. Extrait du jeu de test annoté 13
10.2. Structure du prompt RAG 13
10.3. Exemple de réponse JSON de l'API 13
10.4. Conteneurisation Docker 14

1. Objectifs du projet

1.1. Contexte

Puls-Events est une entreprise technologique qui développe une plateforme de recommandations culturelles personnalisées. Dans le cadre de sa stratégie d'innovation, l'entreprise souhaite intégrer un chatbot intelligent capable de répondre aux questions des utilisateurs sur les événements culturels à venir, en s'appuyant sur les données réelles issues de l'API Open Agenda.

1.2. Problématique

Les approches classiques de recommandation reposent sur des filtres statiques (lieu, date, catégorie) qui ne permettent pas de répondre à des questions formulées en langage naturel. Un système RAG (Retrieval-Augmented Generation) répond à ce besoin en combinant la recherche sémantique dans une base de données vectorielle avec la génération de réponses contextualisées par un LLM. Cela permet à l'utilisateur de poser des questions complexes et croisées (par exemple : « Quels concerts ont lieu à Valence ce week-end ? ») et d'obtenir des réponses pertinentes basées sur des données réelles.

1.3. Objectif du POC

Ce POC (Proof of Concept) vise à démontrer la faisabilité technique et la valeur métier d'un assistant de recommandation d'événements culturels basé sur un système RAG. Il doit prouver que le système est capable de fournir des réponses pertinentes, contextualisées et fondées sur des données réelles, tout en étant exposé via une API REST utilisable par les équipes produit et marketing.

1.4. Périmètre

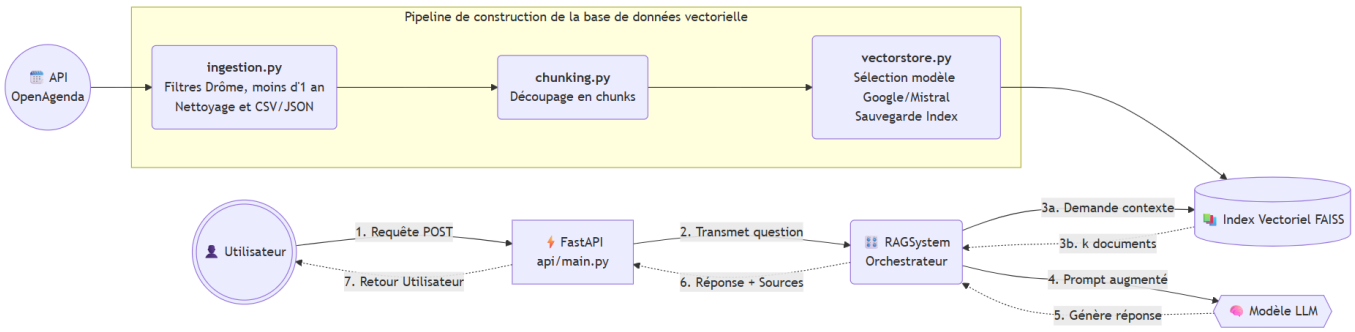
- **Zone géographique :** département de la Drôme (26)
- **Période :** événements des 12 derniers mois
- **Source de données :** API Open Agenda via OpenDataSoft
- **Volume :** 1 068 événements collectés, 1 403 chunks indexés

2. Architecture du système

2.1. Schéma global (UML)

Le diagramme d'architecture ci-dessous (disponible également dans le README du dépôt GitHub au format Mermaid) représente les deux flux principaux du système :

- **Flux de construction :** API OpenAgenda → ingestion → chunking → vectorisation → index FAISS
- **Flux utilisateur :** Question HTTP → FastAPI → RAGSystem (retriever + prompt augmenté + LLM) → Réponse + Sources



Note : Le schéma UML Mermaid complet est intégré dans le README.md du dépôt GitHub sur la page du projet.

2.2. Technologies utilisées

Composant	Technologie
Orchestration RAG	LangChain (LCEL)
Base vectorielle	FAISS (faiss-cpu)
LLM (génération)	Google Gemini 3.1 Flash Lite Preview (fallback de Mistral)
Embeddings	Google gemini-embedding-2-preview (768 dim.) — fallback de mistral-embed
API REST	FastAPI + Uvicorn
Évaluation	Ragas + évaluation manuelle
Conteneurisation	Docker (python:3.12-slim)
Gestion des dépendances	UV (avec export requirements.txt pour Docker)
Source de données	Open Agenda via API OpenDataSoft

3. Préparation et vectorisation des données

3.1. Source de données

Les données proviennent de l'API publique Open Agenda, exposée via la plateforme OpenDataSoft. Le module *ingestion.py* gère la collecte avec les paramètres suivants :

- **Filtre géographique :** département = Drôme (26)
- **Filtre temporel :** événements des 365 derniers jours
- **Pagination :** automatique (100 résultats par page)
- **Volume collecté :** 1 068 événements

3.2. Nettoyage

Le pipeline de nettoyage (intégré dans ingestion.py) applique les traitements suivants :

- Suppression des balises HTML dans les descriptions d'événements
- Déduplication des événements (par identifiant unique)
- Suppression des entrées vides ou incomplètes
- Sélection des champs pertinents : titre, dates, lieu, description, mots-clés
- Structuration de chaque événement en texte formaté pour la vectorisation

Les données nettoyées sont sauvegardées en JSON et CSV dans le dossier data/ pour permettre une utilisation sans appel API.

3.3. Chunking

Le module *chunking.py* convertit chaque événement en Document LangChain avec métadonnées (titre, dates, lieu, URL), puis découpe les textes longs via RecursiveCharacterTextSplitter.

Paramètres de chunking :

- chunk_size = 1 000 caractères
- chunk_overlap = 150 caractères

Justification : l'analyse de la distribution des longueurs des textes a montré que 85 % des événements font moins de 1 000 caractères. Ce seuil préserve l'intégrité de la majorité des événements tout en découpant les 15 % restants en 2-3 chunks avec chevauchement. Le chevauchement de 150 caractères évite la perte de contexte aux frontières entre chunks.

Les métadonnées de l'événement source sont propagées à chaque chunk, ce qui permet d'afficher les informations de l'événement (titre, lieu, dates) dans la réponse finale.

Résultat : 1 068 événements → 1 403 chunks (41 non découpés, 9 découpés en plusieurs chunks).

3.4. Embedding

Modèle utilisé : gemini-embedding-2-preview (Google) en fallback de mistral-embed (Mistral AI). Voir la section 4 pour la justification de ce choix.

- **Dimensionnalité :** 768 dimensions
- **Types de tâche asymétriques (spécificité Google) :** RETRIEVAL_DOCUMENT à l'indexation, RETRIEVAL_QUERY à la recherche
- **Logique de batch :** batchs de 100 chunks avec délai de 65 secondes entre chaque batch pour respecter les quotas (1 000 RPD (Request Per Day, 100 RPM (Request Per Minute))
- **Construction incrémentale :** le script build_index.py supporte la reprise (--resume) pour gérer les quotas journaliers insuffisants pour un build complet

4. Choix du modèle NLP

4.1. Modèle cible : Mistral AI

Le modèle initialement sélectionné est Mistral AI, conformément aux exigences du projet. Mistral présente plusieurs avantages : modèle français performant sur les textes en langue française, compatibilité native avec LangChain, et offre de tarification compétitive pour un usage en production.

4.2. Fallback : Google Gemini

En raison d'un problème d'accès à l'API Mistral (ticket de support ouvert, non résolu au moment du développement), Google Gemini a été utilisé comme fallback :

- **LLM de génération :** gemini-3.1-flash-lite-preview
- **Modèle d'embedding :** gemini-embedding-2-preview

L'architecture a été conçue dès le départ pour permettre le switch entre providers via deux variables d'environnement (LLM_PROVIDER et EMBEDDING_PROVIDER). Le passage de Mistral à Gemini — ou l'inverse — ne nécessite aucune modification de code, uniquement un changement de configuration dans le fichier *.env*.

4.3. Critères de sélection

- **Compatibilité LangChain :** les deux providers disposent d'intégrations LangChain officielles (langchain-mistralai, langchain-google-genai)
- **Qualité en français :** les deux modèles génèrent des réponses fluides et pertinentes en français
- **Coût :** les deux offrent un tier gratuit suffisant pour un POC

- **Limites du tier gratuit Gemini :** 15 RPM (Requests Per Minute) pour le LLM, 1 000 RPD pour les embeddings. Ces limites ont impacté la construction de l'index et l'évaluation Ragas (voir sections 7 et 8).

4.4. Prompting

Le prompt RAG a été itéré sur plusieurs versions pour optimiser la pertinence des réponses. Les éléments clés du prompt final :

- Injection de la date du jour ({today_date}) pour permettre au LLM de distinguer événements passés et à venir
- Instructions pour proposer des alternatives quand aucun résultat exact n'est trouvé
- Consigne de citer les sources (titre, lieu, dates) dans la réponse
- Température fixée à 0.2 pour des réponses factuelles et déterministes

4.5. Limites du modèle

- Le LLM ne peut pas vérifier la fraîcheur des données — il répond uniquement sur la base du contexte fourni par le retriever
- Le format de réponse Gemini (list[dict]) diffère de Mistral (str), nécessitant une fonction extract_text() dans *llm.py*
- Les quotas du tier gratuit limitent les possibilités d'évaluation et de reconstruction de l'index

5. Construction de la base vectorielle

5.1. FAISS

FAISS (Facebook AI Similarity Search) a été retenu comme vector store pour les raisons suivantes :

- Conformité aux exigences du projet (FAISS explicitement mentionné dans la mission)
- Open-source et gratuit, adapté à un POC
- Performant pour des volumes modérés (< 100 000 vecteurs)
- Intégration native avec LangChain (langchain_community.vectorstores.FAISS)
- Pas de serveur externe requis — l'index est un fichier local

5.2. Stratégie de persistance

- **Format de sauvegarde :** FAISS natif via save_local() / load_local() de LangChain (fichiers index.faiss + index.pkl)
- **Emplacement :** dossier faiss_index/ à la racine du projet
- **Versionné dans Git :** l'index est commité pour que l'évaluateur puisse tester le système sans reconstruire l'index (ce qui nécessiterait un quota API)
- **Reconstruction :** possible via build_index.py (standalone) ou l'endpoint POST /rebuild (dans l'API)

5.3. Métadonnées associées

Chaque chunk conserve les métadonnées suivantes de l'événement source :

Métadonnée	Description
title	Titre de l'événement
first_date / last_date	Dates de début et de fin
location_city	Ville de l'événement
location_name	Nom du lieu (N/A dans la plupart des cas — limitation identifiée)
canonicalurl	URL de la page événement sur Open Agenda
keywords_fr	Mots-clés de l'événement

6. API et endpoints exposés

6.1. Framework

L'API est construite avec FastAPI, choisi pour sa documentation automatique (Swagger/OpenAPI), sa validation native via Pydantic, et ses performances asynchrones. Le serveur est Uvicorn.

6.2. Endpoints

Méthode	Route	Description
GET	/	Redirection vers la documentation Swagger (/docs)
GET	/health	Vérification de l'état de santé de l'API et du RAGSystem
GET	/metadata	Informations techniques sur l'index et les providers actifs
POST	/ask	Poser une question au système RAG. Body JSON : {"question": "..."}
POST	/rebuild	Reconstruire la base vectorielle. Paramètre optionnel : ?limit=N

6.3. Exemple d'appel et de réponse

Requête POST /ask :

```
{
  "question": "Quels événements à Valence ?"
}
```

Réponse (200 OK) :

```
{
  "answer": "Voici les événements à Valence ...",
```

```
  "sources": [
    {
      "title": "Festival de Valence",
      "url": "https://openagenda.com/..."
    }
  ]
}
```


6.4. Gestion des erreurs

- **400 Bad Request :** question vide ou invalide
- **500 Internal Server Error :** erreur LLM, index FAISS absent ou corrompu

Le RAGSystem est initialisé une seule fois au démarrage du serveur via le mécanisme lifespan de FastAPI, évitant de recharger l'index FAISS à chaque requête.

6.5. Tests

L'API est testée via 5 tests unitaires mockés (tests/test_api.py) utilisant le TestClient de FastAPI. Les tests vérifient les codes de retour HTTP, le format JSON des réponses, et les cas limites (question vide, erreur LLM). Les mocks isolent les tests des appels API externes.

Au total, le projet compte plus de 100 tests unitaires couvrant l'ensemble des modules : ingestion, chunking, vectorstore, LLM, RAGSystem et API.

7. Évaluation du système

7.1. Jeu de test annoté

Un jeu de 20 paires question/réponse de référence a été construit manuellement, réparti en 5 catégories :

Catégorie	Nombre	Exemple de question
Lieu	3	« Quels événements à Valence ? »
Type	4	« Y a-t-il des concerts dans la Drôme ? »
Date	4	« Que faire ce week-end ? »
Croisé	4	« Concerts à Valence en mars ? »
Hors périmètre	5	« Quel temps fait-il demain ? »

Les réponses de référence (ground truths) ont été vérifiées contre les données source (events_drome.csv). Le jeu de test est disponible dans data/test_dataset.json.

7.2. Évaluation manuelle

Chaque réponse du RAG a été classée manuellement (correcte / partiellement correcte / incorrecte) via le script scripts/evaluate_manual.py :

Catégorie	Score	Analyse
Lieu	100 %	Recherche sémantique très performante sur les noms de villes
Croisé	100 %	Le système gère bien les requêtes multi-critères
Hors périmètre	100 %	Le LLM reconnaît et signale les questions hors sujet
Type	87,5 %	Bonne performance, quelques confusions sur les catégories
Date	37,5 %	Limitation structurelle de la recherche sémantique pure

7.3. Évaluation automatisée (Ragas)

Le framework Ragas a été utilisé pour une évaluation automatisée sur 4 métriques :

Métrique	Score	Interprétation
Faithfulness	0,624	Le LLM reste globalement fidèle au contexte fourni
Answer Relevancy	0,807	Les réponses sont pertinentes par rapport aux questions
Context Precision	1,000	Les documents retournés sont précis (peu de bruit)
Context Recall	0,623	Des informations pertinentes manquent parfois dans le contexte

Note : résultats obtenus en deux lancements successifs en raison des limites de taux du tier gratuit Gemini (15 RPM). Certaines métriques ont nécessité un second lancement pour être calculées.

7.4. Analyse qualitative

Points forts : le système excelle sur les questions de lieu et les questions croisées, ce qui correspond au cas d'usage principal (un utilisateur cherchant des événements dans sa ville). La gestion des questions hors périmètre est également solide — le LLM reconnaît et signale clairement les questions qui ne relèvent pas des événements culturels.

Point faible identifié : les questions temporelles (37,5 %) révèlent une limitation structurelle de la recherche purement sémantique. FAISS effectue une comparaison vectorielle sans comprendre la sémantique temporelle : « ce week-end » est traité comme un concept sémantique, pas comme un filtre de date. Le retriever retourne donc des événements sémantiquement proches mais potentiellement aux mauvaises dates.

8. Recommandations et perspectives

8.1. Ce qui fonctionne bien

- La recherche sémantique est performante pour les questions de lieu et les questions croisées
- L'architecture modulaire permet de switcher entre providers sans modification de code
- Le système est conteneurisé et reproductible (Docker build → run → test)
- Le prompt engineering a permis d'améliorer significativement la qualité des réponses
- La couverture de tests (100+) assure la robustesse du système

8.2. Limites du POC

- **Recherche temporelle :** FAISS ne supporte pas le filtrage par métadonnées (dates). Les questions temporelles échouent dans 62,5 % des cas.
- **Quotas API :** le tier gratuit Gemini (1 000 RPD embeddings, 15 RPM LLM) est insuffisant pour une utilisation en production.
- **Périmètre géographique :** limité au département de la Drôme. L'extension à d'autres régions nécessiterait une stratégie d'indexation multi-région.
- **Fraîcheur des données :** l'index est statique. Les nouveaux événements nécessitent une reconstruction manuelle (via /rebuild ou *build_index.py*).

8.3. Améliorations possibles

Court terme :

- Ajouter un filtre post-retrieval par date pour éliminer les événements périmés du contexte
- Implémenter une recherche hybride (sémantique + filtres structurés sur les métadonnées)
- Résoudre le ticket Mistral pour utiliser le provider cible

Moyen terme :

- Migrer vers un vector store supportant le filtrage de métadonnées (Chroma, Qdrant, Pinecone)
- Automatiser la mise à jour de l'index (cron job ou webhook)
- Ajouter un classifieur d'intention pour distinguer les questions RAG des questions conversationnelles

Passage en production :

- Déploiement sur un cloud (AWS, GCP, Hugging Face Spaces) avec CI/CD GitHub Actions
- Intégration d'un front-end utilisateur (Streamlit, React)
- Monitoring des performances et des coûts en temps réel

9. Organisation du dépôt GitHub

Le dépôt suit une organisation par responsabilité :

Répertoire	Rôle
src/	Code source principal : ingestion, chunking, vectorisation, LLM, RAGSystem
api/	API REST FastAPI : endpoints, modèles Pydantic, lifespan
scripts/	Scripts utilitaires : évaluation manuelle, évaluation Ragas, tests de recherche
tests/	Tests unitaires et d'intégration (100+ tests)
data/	Données collectées, chunks, jeu de test annoté, résultats d'évaluation
faiss_index/	Index FAISS persistant (1 403 vecteurs, 768 dimensions)
docs/	Documentation technique et rapport

Principes architecturaux : un fichier = une responsabilité, logique métier (src/) séparée de l'exposition HTTP (api/), données versionnées pour la reproductibilité, aucune clé API commitée (*.env.example* fourni).

10. Annexes

10.1. Extrait du jeu de test annoté

Catégorie	Question	Réponse de référence (résumé)
lieu	Quels événements ont lieu à Montélimar ?	Événements culturels à Montélimar : ...
type	Y a-t-il des expositions dans la Drôme ?	Expositions en cours et à venir : ...
date	Que faire ce week-end dans la Drôme ?	Événements du week-end : ...
croisé	Concerts à Valence en mars ?	Concerts programmés à Valence en mars : ...
hors_périmètre	Quel temps fait-il demain ?	Je suis spécialisé dans les événements culturels...

Le jeu de test complet (20 questions) est disponible dans *data/test_dataset.json*.

10.2. Structure du prompt RAG

Le prompt utilisé par le RAGSystem suit cette structure :

- Rôle : assistant spécialisé dans les événements culturels de la Drôme
- Contexte : documents retournés par le retriever ({context})
- Date du jour : injectée dynamiquement ({today_date})
- Consignes : répondre en français, citer les sources, proposer des alternatives si aucun résultat exact
- Question de l'utilisateur : ({question})

10.3. Exemple de réponse JSON de l'API

POST /ask

Body: {"question": "Quels événements à Valence ?"}

Réponse 200 OK:

```
{
  "answer": "Voici les événements culturels à Valence...",
  "sources": [
    {"title": "Festival de Valence", "url": "https://..."},
    {"title": "Exposition Art Contemporain", "url": "https://..."}
  ]
}
```

10.4. Conteneurisation Docker

Le Dockerfile utilise une image python:3.12-slim (~150 Mo vs ~1 Go pour l'image complète), un utilisateur non-root pour la sécurité, et exploite le cache Docker en copiant requirements.txt séparément. Le build context est réduit à 6,46 Mo grâce au .dockerignore.

Commandes de déploiement :

```
docker build -t puls-events .
docker run -p 8000:8000 --env-file .env puls-events
```

L'API est alors accessible à http://localhost:8000/docs.